

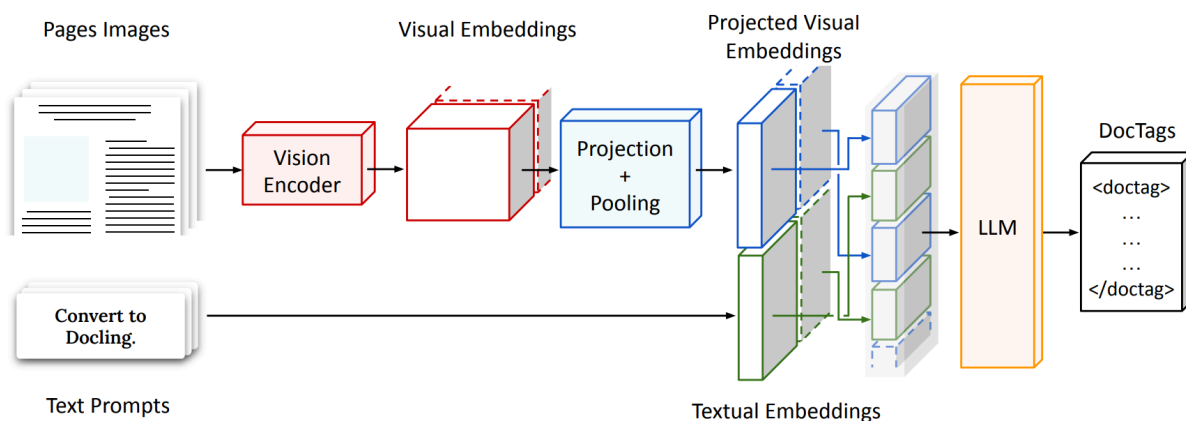
# Public Speaking Coach

## Core Skills:

1. Smol Docling
2. crawl4ai
3. Google Agent Development Kit

## Smol Docling:

1. A small VLM that is finetuned to convert Image+Text Documents to DocTags (A structured format that retains info of images, captions, tables, code, text, location of images).
2. Link: <https://docling-project.github.io/docling/> Use this to tweak Smol Docling when required
3. Paper: <http://arxiv.org/pdf/2503.11576>
4. Architecture:



4. A Generic VLM Architecture:

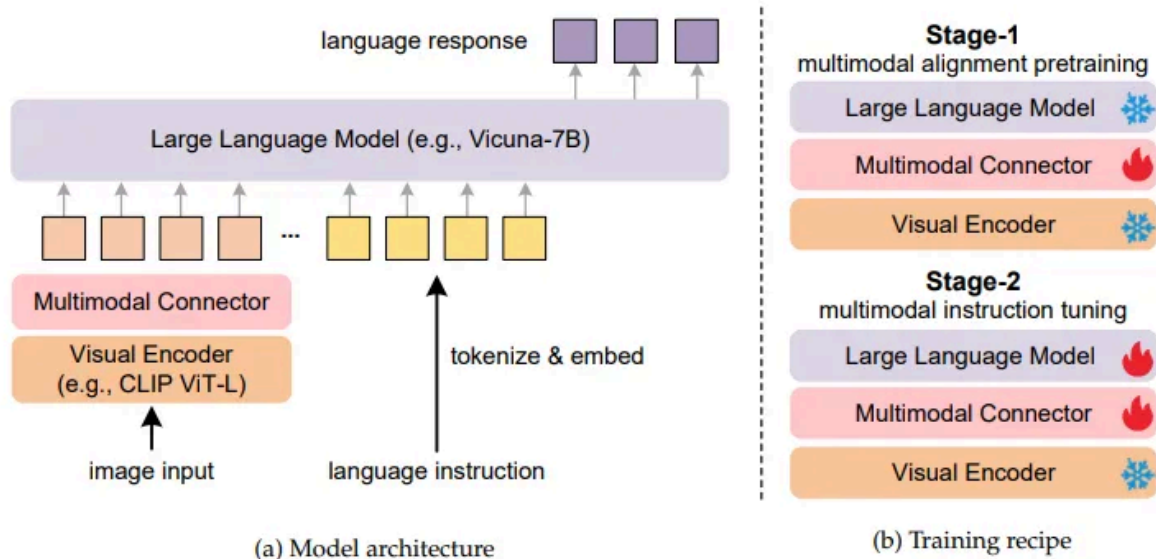


Figure 1 | LLaVA-1.5's model architecture and its two-stage training recipe.

## ADK Custom Agent Class:

Code:

```
class script_creator(BaseAgent):
    """
    Custom agent for a story generation and refinement workflow.

    This agent orchestrates a sequence of LLM agents to generate a story,
    critique it, revise it, check grammar and tone, and potentially
    regenerate the story if the tone is negative.
    """

    # --- Field Declarations for Pydantic ---
    # Declare the agents passed during initialization as class attributes with type hints
    script_creator_agent: LlmAgent
    extract_guiding_info: LlmAgent
    judge1: LlmAgent
    judge2: LlmAgent
    judge3: LlmAgent
    improver_agent : LlmAgent
```

```

note_adder : LlmAgent
fact_checker : LlmAgent

loop_agent: LoopAgent
#sequential_agent: SequentialAgent
parallel_agent: ParallelAgent

# model_config allows setting Pydantic configurations if needed, e.g., arbitrary
model_config = {"arbitrary_types_allowed": True}

def __init__(
    self,
    name : str,
    script_creator_agent: LlmAgent,
    extract_guiding_info: LlmAgent,
    judge1: LlmAgent,
    judge2: LlmAgent,
    judge3: LlmAgent,
    improver_agent : LlmAgent,
    note_adder : LlmAgent,
    fact_checker : LlmAgent,
):
    """
    Initializes the ps_coach.

    Args:
        name: Name of the root agent.
        script_creator_agent: script writer agent
        extract_guiding_info: guiding information extraction agent
        judge1: audience resonance and goal alignment judge agent
        judge2: logic and clarity and structure judge agent
        judge3: factuality and accuracy judge agent
        Correct_Incorp : critical incorporation agent
        note_adder : Adds helper notes to the final script
        fact_checker : checks facts and corrects if needed
    """

```

```

# Create internal agents *before* calling super().__init__

parallel_agent = ParallelAgent(
    name="parallel_agent",
    sub_agents=[judge1, judge2]
)

loop_agent = LoopAgent(
    name="loop_agent", sub_agents=[parallel_agent, improver_agent ], max_i

# Define the sub_agents list for the framework
sub_agents_list = [
    extract_guiding_info,
    script_creator_agent,
    loop_agent,
    note_adder,
    fact_checker
]

# Pydantic will validate and assign them based on the class annotations.
super().__init__(
    name=name,
    script_creator_agent = script_creator_agent,
    extract_guiding_info=extract_guiding_info,
    judge1=judge1,
    judge2=judge2,
    judge3=judge3,
    improver_agent=improver_agent,
    fact_checker = fact_checker,
    note_adder=note_adder,
    parallel_agent=parallel_agent,
    loop_agent=loop_agent,

```

```
sub_agents=sub_agents_list, # Pass the sub_agents list directly
)
```

## **This is definitely an unusual class creation. Here is the explanation:**

1. If we don't use "self. " we can still create variables but they will be local variables. Not accessible from outside like we usually do example: 'cat.age()'
2. Because we declared the type of all the attributes at the very beginning of the class, pydantic knew that these were the attributes that you would input to the base class initialiser. This is why it works even though parent class does not have these attributes declared in its initialiser.
3. Declaring variable types at the beginning tells pydantic which fields to look out for in the parent initialiser. It can be less but it cant be more.